

```

1 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en">
2 <head>
3   <title>Content filters</title>
4   <script type="text/javascript" src="jquery-1.3.3.js"></script>
5 </head>
6 <body>
7   <table>
8     <tr>
9       <td>Article Name</td>
10      <td>Status</td>
11      <td><input type="checkbox" checked="" /></td>
12      <td><input type="checkbox" checked="" /></td>
13      <td><input type="checkbox" checked="" /></td>
14    </tr>
15  </table>
16 </body>
17 </html>

```

Figure 2: Content filters

Note that if content filters are made to filter on `*` then they will select the complete DOM, even if only one DOM element satisfies. For example, `$(table:has(td)).size()` will be 1, but `$(has(td))` will be of size 7 (in the above fragment) since it selects all the above tr(s), the body and HTML too.

Not selectors

We have looked at a good number of selectors till now. In certain scenarios, when we need a function that will negate the available functionality, the `:not` selector comes to the rescue. As we know, we can use `$(checkbox:checked)` to get all the check boxes that are selected. But how about getting the check boxes that are not selected by the user? It's simple! We need to use `:not` in front as `$(checkbox:not('checked'))`.

Selector commands

There are a set of JQuery commands/functions that can be used to select an element. We know that every JQuery selector will return an array of elements. To get a particular element out of the array, we can use `get(number)`. So `$(button).get(0)` is equivalent to `$(button:first)`. The inverse of `get` is `index`, which is used to find the position of an element in an array of elements. Considering a page that has only one `submit`, `element($("submit")[0])` will return the position of the submit button in the complete DOM. When we need to select all input elements that are either of `type radio` or `checkbox` then we can use any one of the following: `$(radio:checkbox)` or `$(radio).add(checkbox)`. Following which, the `add(JQuery expression)` will add the result to the wrapped set.

When we need to remove the element that is

added as part of the previous command from the wrapped set, then we can use `end()`. For example:

```
$(image).add('text').addClass('inputandtextclass').end().add('imageclass')
```

In the above case, the `inputandtextclass` class is applied to both image and text elements, whereas `imageclass` class will be applied to the image elements only.

We use `find(expression)` to select elements that match the given criteria and also the descendants of the already selected element set. We use `filter(expression)` to select elements that match the given criteria from the given set.

```

<div>
  <button value="Check">Check</button>
</div>

```

In the above HTML snippet, to select the button using:

- find, use `$(div).find('button')`. This will search for buttons under `div` element.
- filter, use `$(div:button).filter('button')`. This will search for the button element in the first wrapped set, which has both `div` and `button`.

Although we have equivalent selectors `$(div:button)` and `$(div:button, $(div))`, respectively, `find` and `filter` will be more useful in chaining.

JQuery selectors and filters have made the selection of a DOM element easier. It is always better to use the ID selector since JQuery uses the browser native method `getElementById()`. So in any case, the ID selector is much faster than other selectors. Moreover, it's always better to cache the result of selectors. The cached results can be used (if the DOM element is not updated dynamically) for future purposes, thereby making the response faster. To practice the above concepts, click on the following link: codylindley.com/jqueryselectors. 

By: Lakshmi Narayanan N and Veerabahu Subramanian C

The authors work at HCL Technologies Ltd, Chennai. Lakshmi blogs at grabyourfreedom.blogspot.com and you can contact him at grabyourfreedom@gmail.com. Veerabahu blogs at veechand.wordpress.com and you can contact him at veechand@gmail.com.

itMagz
Information Technology. Today. Tomorrow.

A Portal For Technologists And Technopreneurs